アルゴリズムとプログラミング

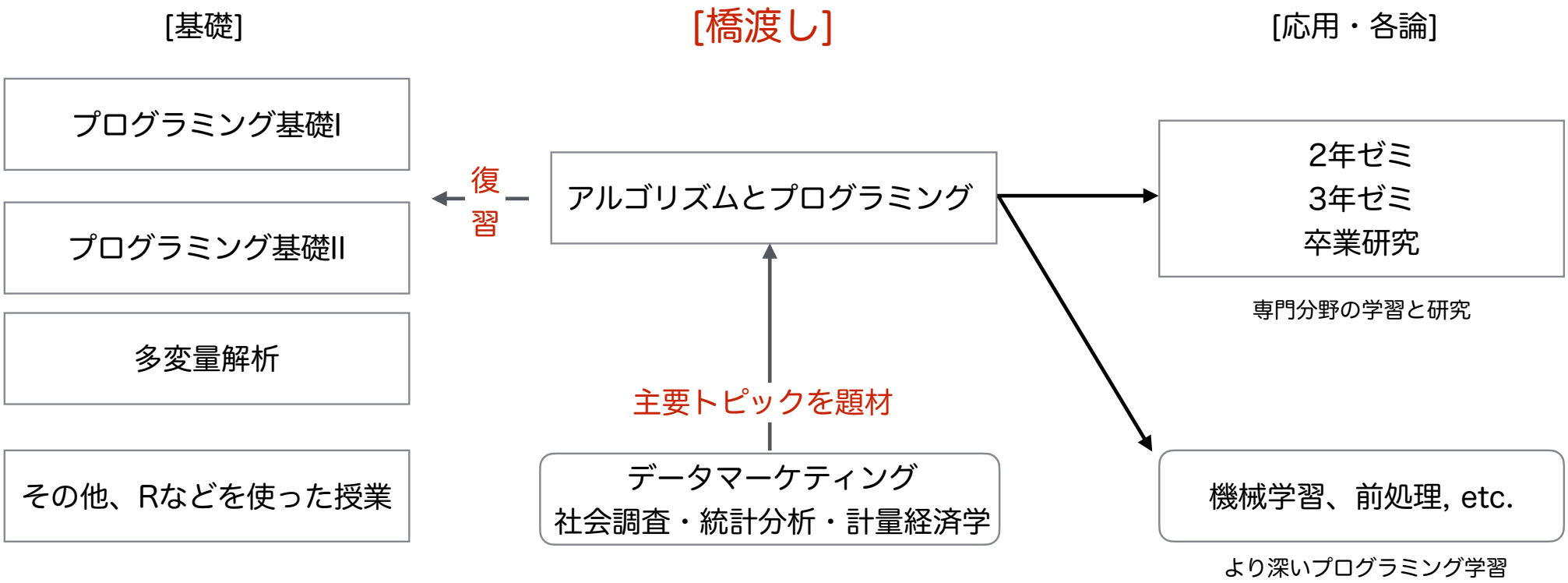
第1回：イントロダクション

担当：田村龍一

本日の予定

- Toyonet-Gに記載されているシラバスは、私がFDSに着任する数年前に、一般的なコンピュータ科学/情報科学系学部学生を想定して記述したものです。
  - この数年の間に、プログラミングに関する教育のスタンダードは大きく変化しました。
  - FDSで展開される授業やゼミで、「どんな局面で、どのようにプログラミングを活用すべきか」が少しずつ固まってきました。
- この授業回では、FDSの現状を踏まえた「改訂版」を提示します

カリキュラムにおける、この授業の位置付け



- 橋渡し役として、この授業の使命
  - プログラミングに関する知識と、FDSの活動で必要となるスキル（**データ構造とアルゴリズム**）を**統合**すること
  - ストレスの少ない、日常的に使える**プログラミング環境を整備**すること。

この授業の特色

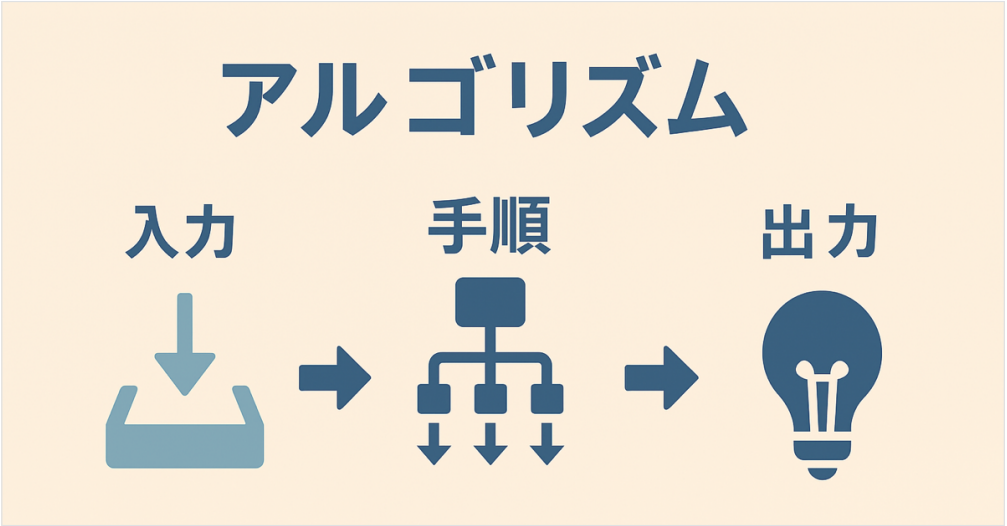
- **データサイエンス・フードシステム＆経済実証分析**で使われるプログラミングスキルにこだわる
  - これらの分野における、2大プログラミング言語：**Python と　R、両方を対象とする。**
- プログラミング作業を**ストレス少なく行える学習環境**にこだわる
  - WordやExcelではなく、プログラミング作業に最適化された「**テキストエディタ**」の使用が不可欠。
  - この授業で用いるテキストエディタ＝**Visual Studio Code (VS Code; 統合環境)**
  - **Jupyter Notebook教材**を用いた「ステップバイステップ方式」によるプログラム実行確認
- **生成AI** (ChatGPT・Copilot・Geminiなど）との**共同作業スタイル**を身につける
  - プログラミング時のエラー処理から、コーディング補助まで、学習の伴侶として十分に役立つ。
  - 自己能力開発ツールとして生成AIを活用するためのプロンプトを身につける

どうしてPythonとRを、同時に勉強するのか？

- プログラムを書いて**問題解決するロジックは、言語によらず共通**している。このロジックのうち、最も大事なのが「**アルゴリズム**」
  - アルゴリズム＝プログラムによる計算で問題解決する典型的な作業の道筋(**ロジック**)
  - 世の中のプログラムのほぼ全ては、アルゴリズムを意識して記述されている。生成AIも、アルゴリズムを反映したプログラムコードを作り出す。
  - 言語の違いは、アルゴリズムを表現する上での、方言の違いにしかならない！
- ただし、PythonとRには得意分野と不得意分野がある。
  - 伝統的な経済統計分析機能は、Rの方が圧倒的に充実している
  - 自然言語処理(口コミ分析など)、機械学習は、Pythonが世界標準。
  - 解決したい課題に応じて言語を使い分けできることが、FDSの学生として大事！

アルゴリズムとは

- 問題を解くための手順や方法を、順序立てて明確に記述した道筋・ロジック。→ コンピュータにとっての「料理レシピ」のようなもの。
- 特徴
  - 入力(データ)があり、出力(中間データや分析結果)が得られる
  - 曖昧さがなく、誰が実行しても同じ結果になる
  - ステップバイステップの手順で構成される。
- プログラムで典型的に使われる局面
  - 分析の順番や実行順序を制御する
  - 複数データを統合したり、必要なデータを取り出す
- 人間が見通しよく分析できる手順であるため、データを使った課題の発見や解決が容易になる。



典型的なアルゴリズム

探索
ソート(並び替え)
最短経路

DS特有なアルゴリズム

Tidy(整形)
SAC(集計)
Map-Reduce(分割集約)

授業の進め方

- 対面とオンデマンドで構成
- **対面授業(13回)：**用語・考え方・ロジックを学ぶ授業回
  - ロジックを小さなプログラムの断片で言語化し、動作確認
  - PCを持参してください。
- **オンデマンド(2回)：**
  - RとPythonを同時に扱える「プログラミング環境」の設定
  - 生成AIとの共同作業で、データサイエンス作業を完結する

みなさんの授業への取り組み方

- プログラミング開発環境は、プログラミングを学ぶ際の「文房具」のようなものです。この環境に馴染むようにしてください
  - オンデマンドでは、**Visual Studio Code(テキストエディタ＋統合開発環境)という"文房具"**を構築し、PythonとRを実行する演習が含まれています。
- 対面授業回では、アルゴリズムやデータ構造の**概念・用語・ロジックを覚える**ことに集中します。
  - 理解が追いつかなかったとしても、どの授業回で何が紹介されたかくらいは覚えておきましょう。
- オンデマンド授業回では、生成AIと一緒に**プログラムを構築する**
  - プログラミングは、結局は習うより慣れろ、の世界です。生成AIの力を借りながら自分の手でプログラムを書く経験を積めば、小さな子どもでも身についてしまう内容です。そもそもそんなに大変なタスクではありません。

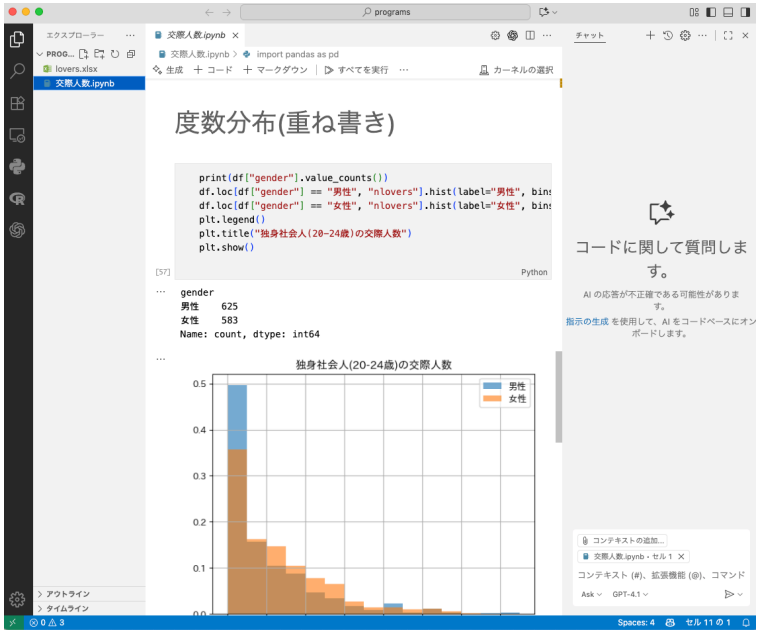
プログラミングを学ぶためのツール

プログラムを書く、実行する、ミスを直す、出力結果を検討する、品質改善する
このような一連のタスクを、できる限り見通しよくスムーズに行いたい

プログラムを書きやすいアプリ："テキストエディタ"

プログラムを実行しやすいアプリ："実行環境"

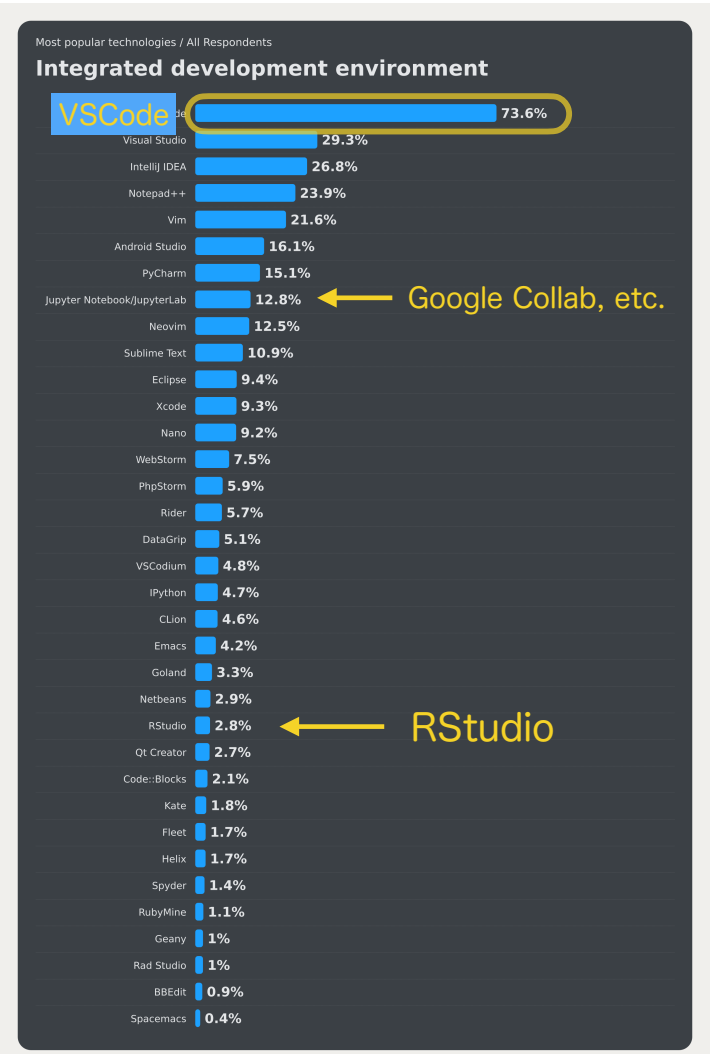
「統合開発環境(IDE)」



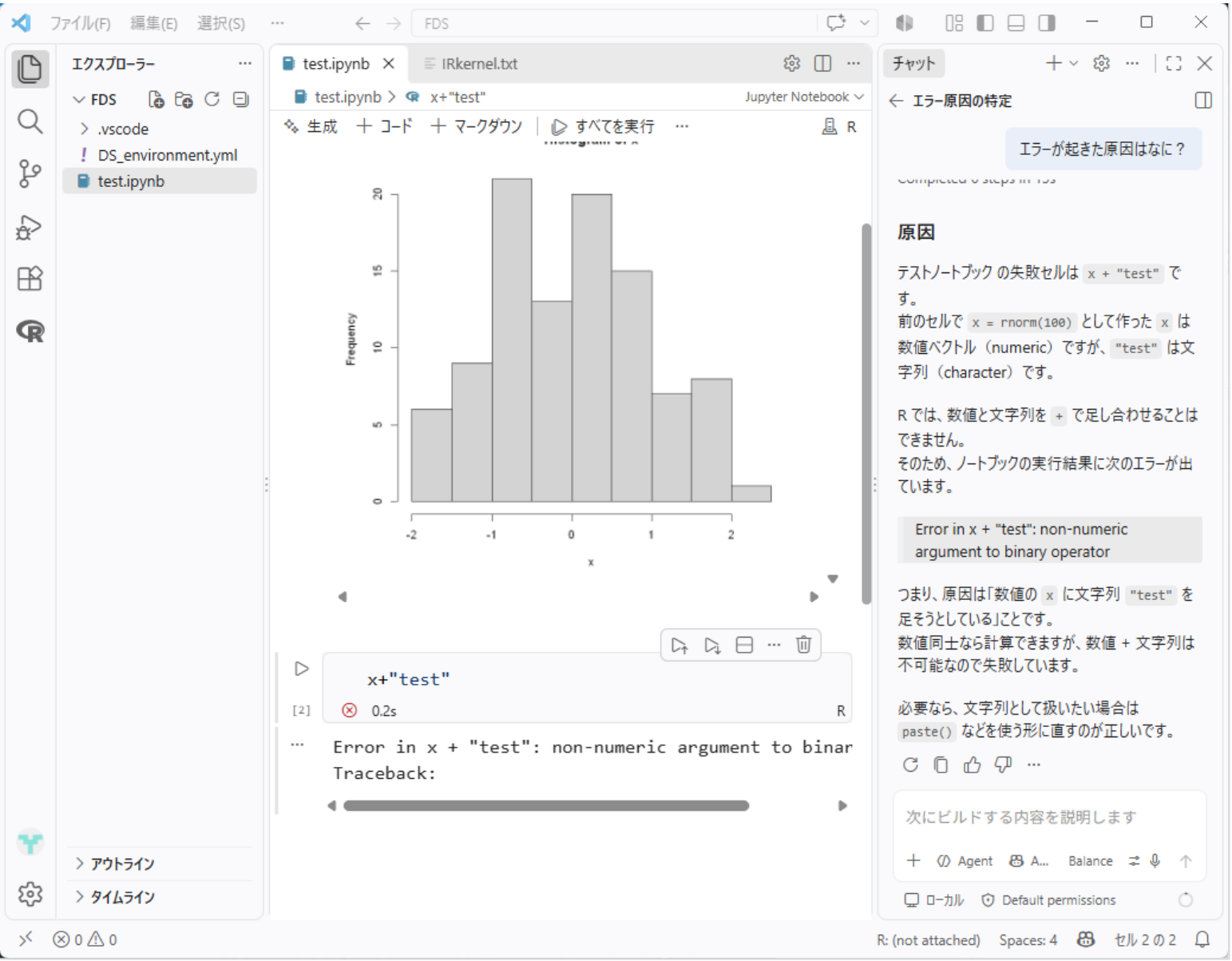
Visual Studio Code
(by Microsoft)

VSCodeは、"世界標準"の統合開発環境

- VSCodeは、プロフェッショナルから学び始めの人たちまで広く使われている、テキストエディタである。
- 2024年に世界最大のオンラインプログラマーコミュニティ**Stack Overflow**が行ったサーベイによると、ユーザーの73.6%がVSCodeを使っている。
  - Google Collabなどで使われるJupyter Notebook/Labは、12.8%。
  - RStudioはもっぱらRプログラミングにしか使われないため、利用者シェアは2.8%程度
- この授業では、VSCodeとJupyterを組み合わせた快適な"プログラミング環境"を構築し(→ 第2回オンデマンドの内容)、勉強をすすめていく。



https://survey.stackoverflow.co/2024/technology#most-popular-technologies-new-collab-tools



- VSCodeでRプログラムを作成中
- 画面右側には、Github Copilotという生成AIが常駐している。
  - 事前ユーザー登録が必要
  - 別の生成AI(ChatGPT)を使うこともできる。
- 学習中に、エラーが発生したときは、チャットに相談してエラー原因と解決策を教えてくれる。
  - 修正中に、生成AIが「うまく動くコード」リアルタイムに補完してくれる

オンデマンド・パート1

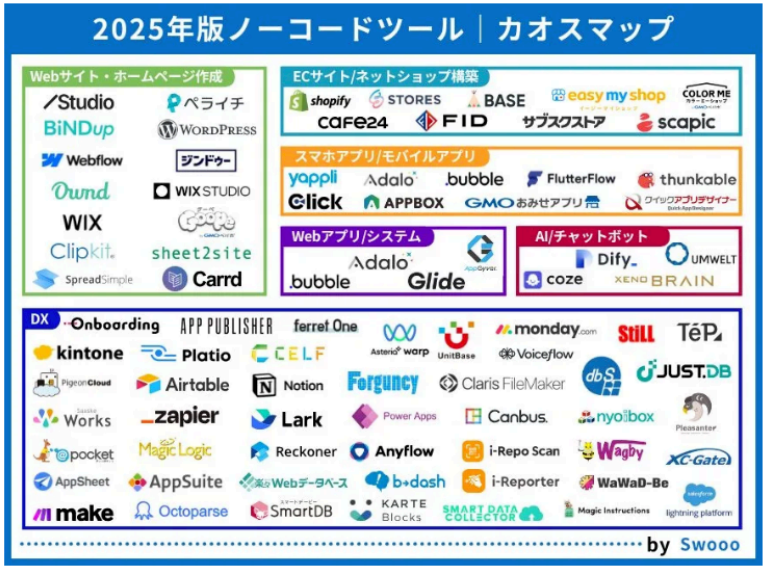
今週中に、ACEでアナウンスします

- プログラミング環境「VSCode」のインストール
  - この授業やFDSの他の授業で使うことになる、RとPythonライブラリを一括インストール
- Github Copilotのユーザー登録

AI時代に求められるプログラミングスキルとは何か？

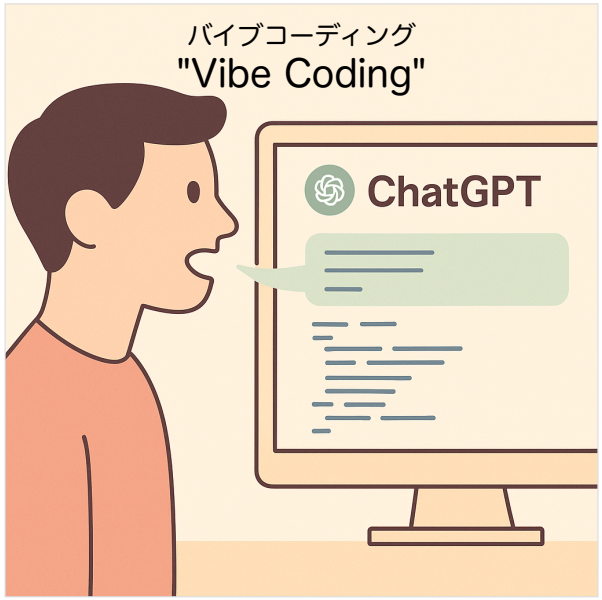
授業スケジュール
成績評価の方法と、根拠

背景：ノーコードツール/バイブコーディング



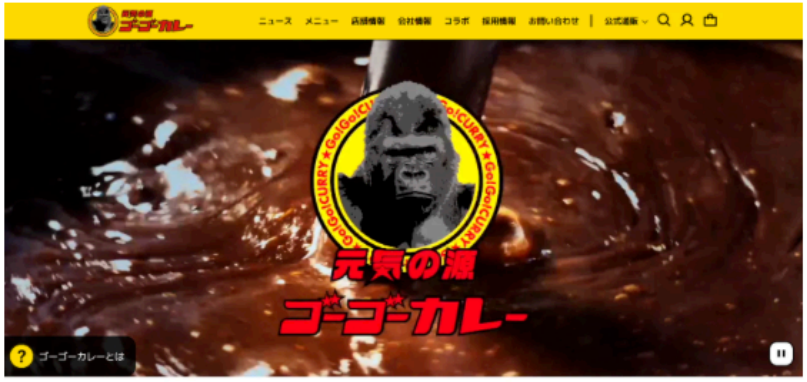
プログラミングが
できなくても
Webサイトや
eコマースなど
オンラインサービスを
作れる！

生成AIとの対話で
プログラムコードを
自動生成する



- 最近は、**AIがプログラムコードを書く時代とも言われている**
  - ノーコード開発・"バイブ(雰囲気)コーディング"、など。

「ノーコード」サービス：Shopify、SNS



ゴーゴーカレーのShopifyストアは、ブランドの力強くユニークな個性を際立たせるデザインが魅力です。黄色と黒を基調としたカラースキームが活気あふれるブランドイメージを作り出しています。

トップページでカツを揚げている様子やカレーを盛り付ける工程を動画で紹介して、読者にいち早く食品の魅力を伝えている点も注目すべきポイントです。これらの工夫により、ボリューム感あふれるカレーの魅力が存分に伝わるデザインとなっています。

出典：Shopify



出典：X

ECサイトや小規模飲食店では、ホームページ制作のためのスタッフを雇うほど余裕がないところが多い。
外注するととてもお金がかかる → Webプログラミングのノウハウがなくても、店舗情報を入力するだけでホームページが作成でき、AIによる売上・顧客分析も可能である(Shopify)。

バイブコーディング

人間が、自然言語で
"アバウト"に指示する

ラーメン二郎の開店情報を自動的に取得するウェブスクレイピングプログラム作って。

思考時間: 14s

了解です。
「ラーメン二郎の開店情報（営業開始・臨時休業・営業時間変更など）を自動取得」するための、設定ファイル駆動型のウェブスクレイピング＆RSS監視スクリプトを用意しました。
Web参照はできないため、実在ページのセレクタは仮置きですが、config.yaml に URL とセレクタを追記するだけで動きます。Slack通知・キーワード絞り込み・重複除去・永続ストレージあり。

生成AIがプログラムを作る

watcher.py

python コピーする

```python
import os
import re
import json
import time
import hashlib
import logging
from datetime import datetime
from typing import Dict, Any, List, Optional

import requests
from bs4 import BeautifulSoup
import feedparser
import yaml
from dateutil import parser as dateparser

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s"
)

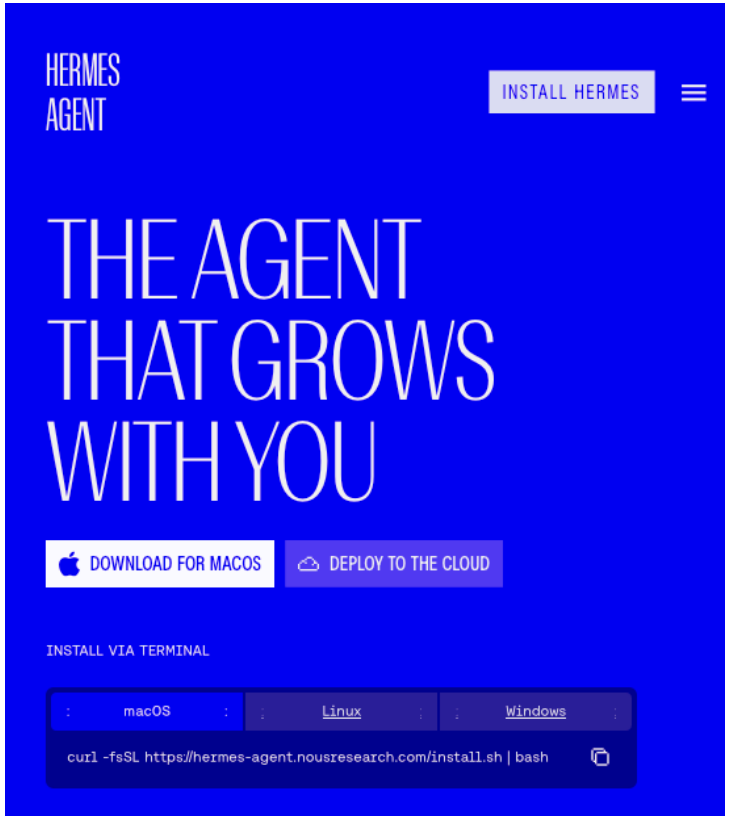
HEADERS = {
    "User-Agent": "jiro-watch/1.0 (+https://example.com; contact: admin@example.com)"
}

STATE_PATH = os.path.join(os.path.dirname(__file__), "state.json")
CONFIG_PATH = os.path.join(os.path.dirname(__file__), "config.yaml")

SLACK_WEBHOOK = os.getenv("SLACK_WEBHOOK_URL")  # 任意: Slack通知先

def load_config(path: str) -> Dict[str, Any]:
    with open(path, "r", encoding="utf-8") as f:
```

日常業務でも、人間とAIの共同作業はふつうになりつつある



ヘルメス・エージェント

- スケジュール/TODOの優先順位管理
- 1日100通以上のメールのやりとり
- 1日数本のミーティング
- 人間はAIを秘書として使う
  - 業務の多くを代理(エージェント)作業させる
  - AIが判断に迷う局面では、人間が意思決定する
  - 人間の指示を学習して、さらに優秀な秘書として育つ

AIがプログラムを書く時代に、私たちは何を学ぶのか？

前提

- 生成AIは、RやPythonのコードを短時間で作ることができる。
- しかし、コードが動くことと、分析が正しいことは同じではない。

求められること

AIが作ったコードと分析結果を、自分で確かめ、説明できるか？

必要なスキル

① **コードを「読める」**
何を入力し、どのような処理をして、
何が出力されているのかを追える。

② **コードを「動かして確かめる」**
R / Pythonを実行し、
入力と出力の対応を確認できる。

③ **コードを「書き換えられる」**
変数・条件・分析方法を変更し、
自分の問題に合わせて修正できる。

④ **結果を「判断できる」**
統計量・回帰結果・予測精度などを読み、
その分析が妥当かを判断できる。

コードが動くことと ≠ 分析が正しいこと

- プログラムは、与えられた課題や分析方針にしたがって、処理を実行する。
  - ところが、その課題設定や分析方針そのものが適切かどうかは、プログラムには判断できない。
- だからこそ、分析全体を設計し、結果をどう使うかを判断する責任は人間側にある。
  - 間違った問いや分析方針を与えても、プログラムはその指示に忠実に答えを返す。

人間と、プログラム/AIの共同作業の本質

計算処理はプログラム／AIに任せられる。
何を計算させ、その結果をどう使うかの責任は、人間にある。

授業では、「データ」を起点としてスキルを学びます

FDSの授業やゼミで扱うことが多いタイプのデータを、重点的に取り扱います

データ	例えば…	そこから学ぶこと
市場データ	農産物価格・売上・時系列データ	データ整形 → 可視化 → 回帰分析 → 予測
調査データ	アンケート・社会統計	集計 → クロス分析 → 回帰分析 → 計量経済分析
実験データ	商品開発・官能評価	平均の比較 → 仮説検定 → 多変量解析
商品属性データ	味・価格・成分・商品特徴	相関 → PCA → クラスタリング → 分類・予測

データ → 問い → データを整える → 分析方法を選ぶ → R / Pythonで実行 → 結果を解釈する

成績評価方法

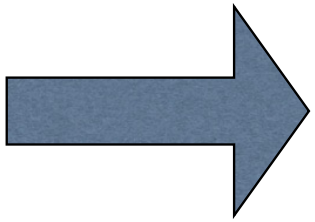
必要なスキル

① コードを「読める」
何を入力し、どのような処理をして、
何が出力されているのかを追える。

② コードを「動かして確かめる」
R / Pythonを実行し、
入力と出力の対応を確認できる。

③ コードを「書き換えられる」
変数・条件・分析方法を変更し、
自分の問題に合わせて修正できる。

④ 結果を「判断できる」
統計量・回帰結果・予測精度などを読み、
その分析が妥当かを判断できる。



- ① 言語の種類を問わず、必要なプログラミング知識
  - データの型・構造
  - 集計・記述分析の作業の流れ
- ② 前スライド4種類のデータに即応した、分析作業の流れ
- 以上の内容を問う、対面式のペーパーテスト(筆記式)を、期末試験で出題する。

成績評価のその他

- 出席：成績評価するかどうかの判定のみに用いる。
  - 6回以上欠席→成績評価の対象にならない：「*」
- 秋学期定期試験ウィークに、**ペーパー試験を実施 → 成績評価**
  - 選択・穴埋め式が8割：資格試験などに類似(授業に現れた用語・アルゴリズムの記憶と理解を問う問題)
  - 記述式が2割：プログラム実行結果を読み解く理解の程度を問う問題
- オンデマンド授業回(第6回終了時に予定)では、ACEのアップロードフォームに成果物提出する。